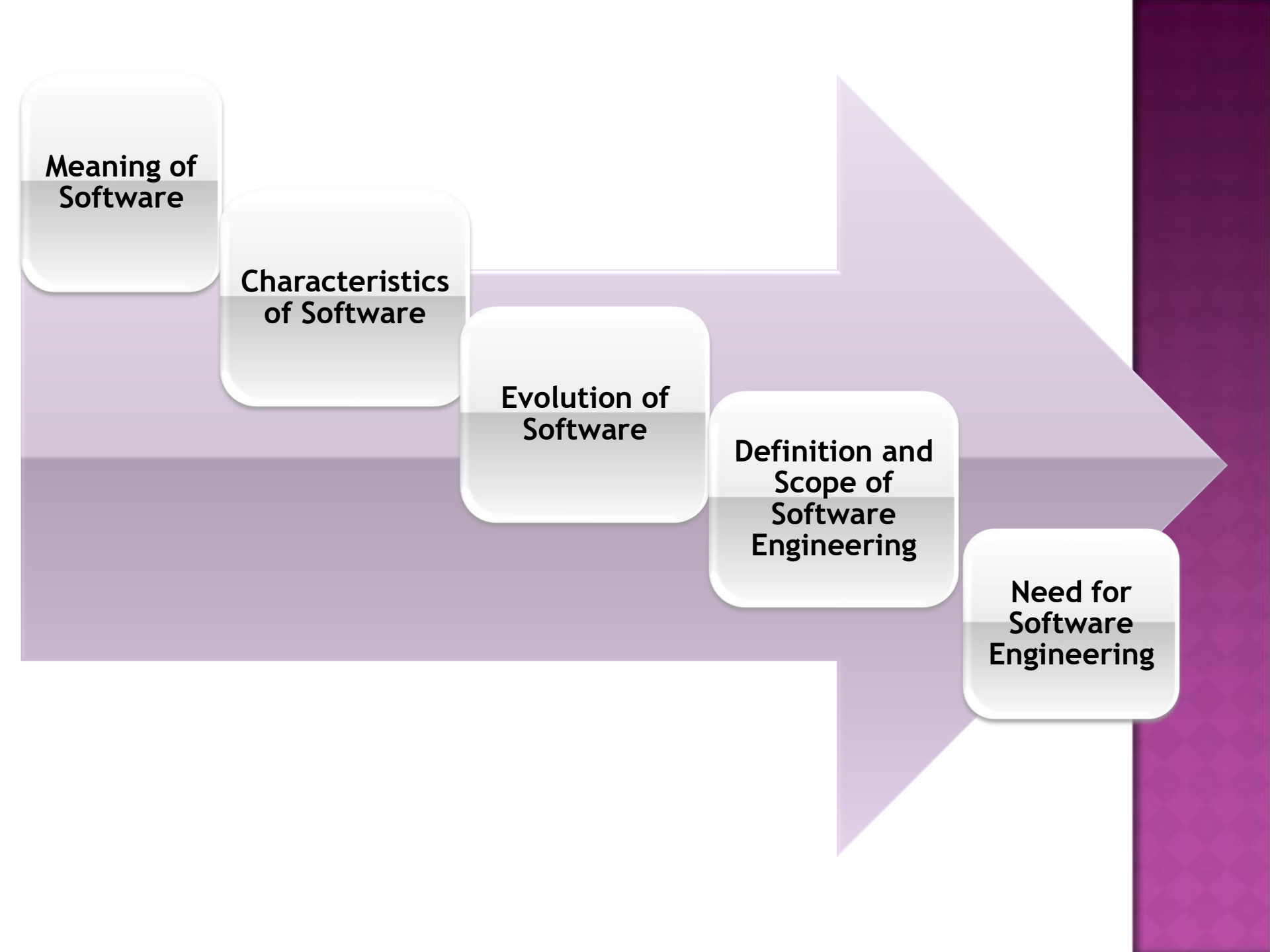


Introduction to Software & Software Engineering



By
Farhan M. Shaikh
B.Sc.(CS), M.Sc.(IT), MCA, B.Ed., M.Ed., UGC-NET(Edn.), M.A.(Sociology)
Best Educationist Award (IIE, New Delhi)



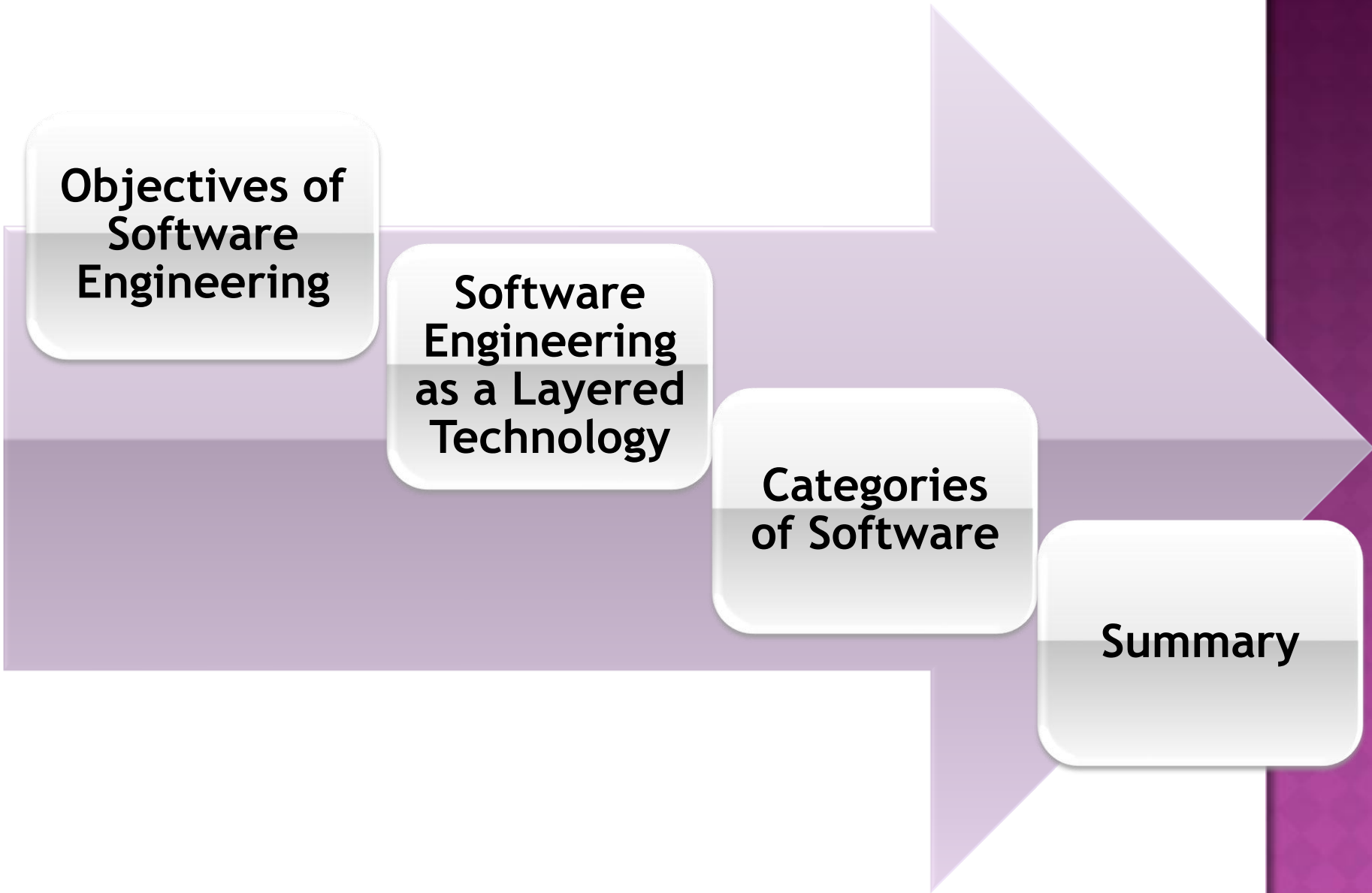
**Meaning of
Software**

**Characteristics
of Software**

**Evolution of
Software**

**Definition and
Scope of
Software
Engineering**

**Need for
Software
Engineering**



**Objectives of
Software
Engineering**

**Software
Engineering
as a Layered
Technology**

**Categories
of Software**

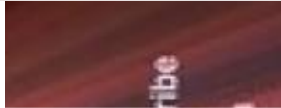
Summary

MEANING OF 'SOFTWARE'

The term *software* refers to the set of computer programs, procedures, data structures, and associated documentation that enables a computer system to perform specific tasks. Unlike hardware—which is tangible and physically perceivable—software is an intangible logical component that dictates how the hardware should behave.



Software can be understood at three interconnected levels:



a) Program Level

A program is a sequence of instructions written in a programming language that directs the computer to perform computations.

b) Data Structures

These define the logical organization of data that programs operate upon. They influence efficiency, scalability, and reliability.

c) Documentation

Documentation explains the design, requirements, and use of the software. Academic research consistently emphasizes documentation as a crucial part of software quality.

Thus, software is not merely "code"; it is a complete system of logic, structure, and communication that supports computational activities.

CHARACTERISTICS OF SOFTWARE



Intangibility: Software cannot be touched or visualized in a physical sense; it exists only as logical constructs



Complexity: Even small programs may contain intricate interactions, making software inherently more complex than many engineered artifacts.



No wear and tear: Software does not degrade physically over time. However, software may require updates and maintenance due to changing environments (known as “software aging”).

CHARACTERISTICS OF SOFTWARE



High Flexibility: Software can be modified relatively easily without altering hardware. This adaptability sets software apart from traditional engineering products.



Dependency on Human Error: Software failures often arise from logic errors introduced during development, not from physical deterioration.

These characteristics justify the need for a formalized discipline—software engineering.

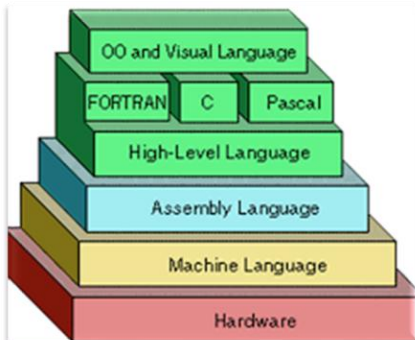
EVOLUTION OF SOFTWARE

```
01001010101
10110010101
10001110100
110010101011
110101010101
```

Generation 1: Machine-Level Software (1940s-50s): Programming was done in binary instructions. Highly error-prone, difficult to maintain.



Generation 2: Assembly & System Software (1950s-60s): Assembly languages and the first operating systems emerged.



Generation 3: High-Level Languages (1960s-80s): FORTRAN, COBOL, C enabled structured programming and large-scale systems.

EVOLUTION OF SOFTWARE



Generation 4: Application & Database Software (1980s-2000s): GUI applications, relational databases, and enterprise systems emerged.



Generation 5: Internet & Agile Era (2000s-present): Web applications, cloud computing, AI-driven systems, DevOps, and Agile/Scrum practices dramatically transformed development processes.

DEFINITION AND SCOPE OF SOFTWARE ENGINEERING

“The systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software.”

(IEEE Standard 610.12)

It is an engineering discipline applied to software, focusing on:

- **Process frameworks**
- **Quality assurance practices**
- **Structured methodologies**
- **Lifecycle management**
- **Risk mitigation**
- **Cost estimation and control**

WHY “ENGINEERING”?

Because software projects require:

- **Planning**
- **Standards**
- **Models**
- **Quality control**
- **Measurement**
- **Management**

—concepts borrowed from traditional engineering and adapted for software.

NEED FOR SOFTWARE ENGINEERING

Concrete problems—budget, schedule, quality, complexity, and maintenance—driving the need for engineering approaches



Budget Overruns: projects exceeding budgets by 50–200%



Schedule Failures: projects often delivered late or abandoned



Poor Quality Software: defects causing failures, losses, hazards



Increasing Complexity: critical systems requiring near-zero failure



Maintenance Burden: 70–80% of cost in maintenance, not development



Software engineering emerged to address these through scientific methods

Historically, software development faced severe challenges known as the **Software Crisis**:

OBJECTIVES OF SOFTWARE ENGINEERING

Key goals that ensure software is high-quality, delivered on time, and easy to evolve



Deliver high-quality software: reliable, efficient, safe, maintainable, secure



Complete projects within time and budget with predictable processes



Improve productivity via standardized workflows and tools



Reduce complexity using models, abstractions, architecture patterns



Support reuse through component-based development



Ensure maintainability for easier updates and extensions

SOFTWARE ENGINEERING AS A LAYERED TECHNOLOGY

How Process, Methods, Tools and Quality work together to deliver engineered software

Layer 1: Process

foundation with SDLC, Agile, Scrum, Waterfall



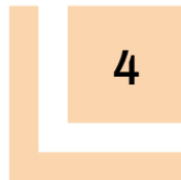
Layer 2: Methods

design principles, modeling techniques, coding standards



Layer 4: Quality Focus

assurance, measurement, verification & validation



Layer 3: Tools

CASE tools, IDEs, version control, automated testing



This layered view demonstrates that engineering requires both **methodology** and **discipline**.

CATEGORIES OF SOFTWARES

Core System & Platform

System Software: OS, device drivers

Embedded Software: microcontroller-based programs

Real-Time Software: flight control, automotive systems

Application & User-Facing

Application Software: business and productivity apps

Web Applications: e-commerce, online services

Mobile Applications: Android, iOS apps

Specialized & Emerging

Engineering & Scientific Software: simulations, CAD systems

AI-based Software: machine learning, neural networks

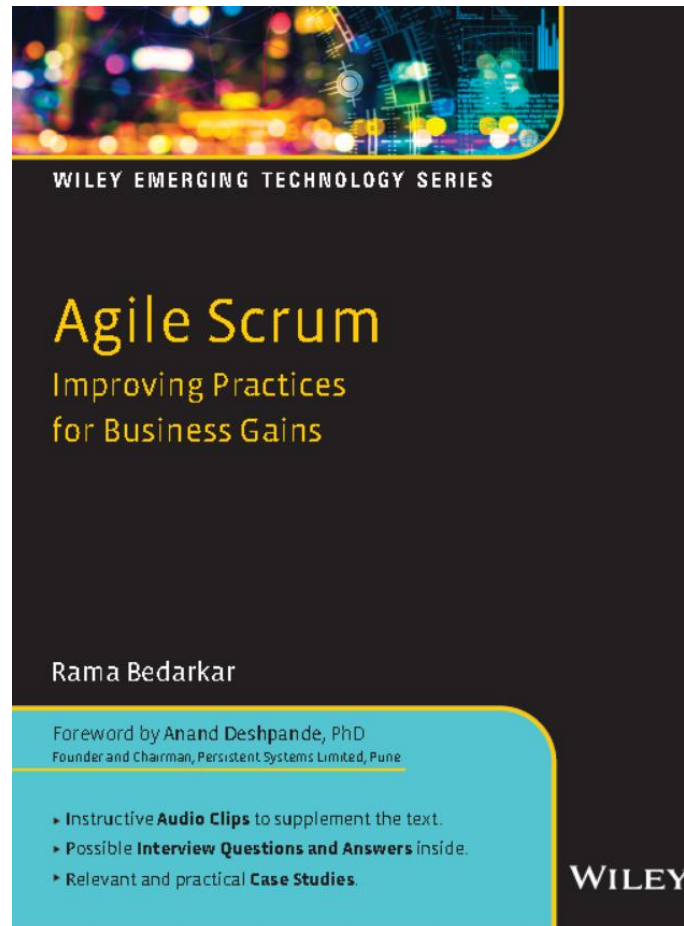
Each category has unique development and quality requirements

SUMMARY

Software Engineering is a structured discipline that addresses complexity, quality, cost, and reliability issues inherent in software development. It builds a bridge between theoretical computer science and practical engineering methods. Module 1 begins with these foundations because Scrum and Agile can only be understood properly when students appreciate *why* structured engineering approaches are essential.

LITERATURE

- “Agile Scrum”, Rama Bedarkar, Wiley, 1/e, 2020



thank
you